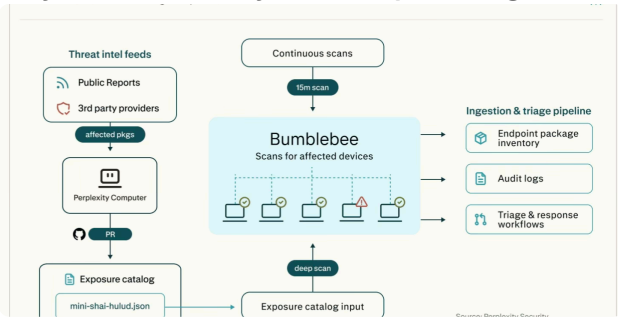
Saturday, 24 October 2026

PURPOSE	EXECUTION MODEL	SPEED	ECOSYSTEMS
Endpoint inventory	Read-only metadata	Fast (seconds)	10+ ecosyst
SBOM generation	Read + heuristics	Medium (30s-2m)	20+ ecosyst
Vulnerability scanning	Read + CVE DB	Medium (1-3m)	10+ ecosyst
Project vulnerability scan	Executes npm	Slow (project-level)	npm only
Python vulnerability scan	Executes pip	Slow (project-level)	PyPI only
Endpoint visibility	Read + system queries	Fast (queries)	General OS, package sup
Vulnerability scanning	Read + CVE DB	Medium (1-2m)	10+ ecosyst

Package analysis

dynamic analysis for package installations



Pip canary:

Security Model

PipCanary does the following:

- It installs packages in a sandboxed environment (using bubblewrap) and tries to load all installed packages inside the sandboxed environment.
- It scans the activities inside the sandboed environment for potentially malicious file system access (using strace).
- If it detects potentially malicious file system access, it kills all processes in the sandboxed environment and reports its findings.
- All packages get removed immediately after scanning.
- After scanning, it consults the PyPI JSON API for known vulnerabilities of all installed packages and reports its findings.

Note that all of the precautions offer better security than running pip install alone "unprotected" but they do not guarantee absolute security for the packages being scanned or the scanning process itself!

Examples:

- The sandboxed environment has network access to the host

machine during the installation process, so it is **not advised to run it inside a network with access to sensitive systems or components**!

- If a malicious packages postpones it's malicious activities after module loading, PipCanary has no chance of detecting this! **PipCanary does not contain a static source code scanner**!

Conclusion

There are natural limitations to all checks PipCanary performs so **running PipCanary is no guarantee for perfect security**. As such PipCanary (as well as any other security tool) can only be a **part of a wider security strategy**!

Similar findings:

https://github.com/DNSZLSK/muad-dib

https://github.com/pypa/pip-audit

https://github.com/DataDog/guarddog

Further Information on PyPi Suppy Chain Attacks

OWASP Top 10 2025

- OWASP Top 10 2025: A03 Software Supply Chain Failures
- OWASP Top 10 2025: A08 Software or Data Integrity Failures

Incidents

- PyTorch Lightning and Intercom-client Hit in Supply Chain Attacks to Steal Credentials
- Incident Report: LiteLLM/Telnyx supply-chain attacks, with guidance

one of the **pip canary** prevention tool is :

enforcing cool down periodd on new package versions (time to identify)

feats by pip canary:

behavioural analysis: package installation and loading using strace and bubblewrap sandboxing

upload time checks

known vulnerability checks

their motto (no difference between a *canary* and a *source code)*.

no unnecessary impediments

***t*rade off for our project:**

Note that all of the precautions offer better security than running pip install alone “unprotected” but they do not guarantee absolute security for the packages being scanned or the scanning process itself!

perplexitys bumblebee:

open source tool for scanning developer machines for supply chain compromises.

when a security advisory names a vulnerable package which developer machines in ur org have it installed right now ?

Unlike SBOMs (Software Bill of Materials) that document production artifacts, or EDR tools that monitor what's running, Bumblebee focuses on the messy local development state scattered across lockfiles, package manager caches, IDE extensions, browser add-ons, and Model Context Protocol (MCP) server configs. In a landscape where supply chain attacks like the 2024 XZ Utils backdoor, 2025 PyTorch supply chain compromise, and ongoing

npmlware campaigns dominate headlines, Bumblebee offers security teams a fast, read-only, zero-dependency tool to assess exposure across their entire developer fleet

supply chain visibility gap:

when some critical advisory drops, what are immediate runs?

which developer have the vulnerable package installer?

which projects are affected?

what versions are in use

traditional tools failures:

SBOMs document production builds, not local dev environments

- **EDR monitors running processes, not dormant dependencies**
- **Package manager queries (npm ls, pip show) are slow, resource-intensive, and require execution**
- **Vulnerability scanners focus on project-level analysis, not fleet-wide inventory.**

(Execution Overhea)-not needed by bumblebee

bumblebee only reads:

lockfiles

package manager extensions

extension manifests

mcp host configs

Bumblebee is a single Go binary built with only the Go standard library. No vendored dependencies, no third-party packages, no supply chain risk in the tool

This makes it trivial to:

- Deploy across heterogeneous fleets (macOS, Linux, ARM, x86)
- Audit the source (minimal codebase, no transitive dependencies)
- Build reproducibly (no dependency resolution surprises)

source by bumblebee:

Limitations and Future Roadmap

Current Limitations

- **No source file analysis: Doesn't parse Python imports, JavaScript requires, or Go imports**
- **No runtime detection: Doesn't monitor what's actually running (use EDR for that)**
- **Exact version matching only: No CVE database, no version range queries (yet)**
- **Limited config formats: MCP support is JSON-only (no TOML, YAML)**
- **No Windows support: macOS and Linux only (Windows support planned)**

Roadmap (as of v0.1.1)

- **Version range matching: Query catalogs with version ranges (e.g., >= 1.2.0 and < 2.0.0)**
- **CVE database integration: Optional CVE lookup for discovered packages**
- **Windows support: Native Windows path handling and registry queries**

- **Container image scanning: Docker/OCI image layer analysis**
- **Config format expansion: TOML, YAML support for MCP configs**
- **Performance improvements: Incremental scanning, filesystem watchers**

specs by bumblebee where I can find scope to fill out gaps they lowered parallel in other outlets for that specs:

Ecosystem filtering (scanning only ecosystems to reduce runtime)

max duration enforcement (set hard time for deep scans on large filesystems

root preview(find which roots will be scanned without actually being scanned)

transport options(https post output)

selftest(run embedded to verify bumble bee detection capability)-builtin

what is speciality of bumblebee?

Fast, read-only, **fleet-wide exposure checking** when you already know what you're looking for (exposure catalogs).

parallelization:

bb uses goroutines for concurrent directory traversal and parsing , scaling to available CPU coOverheane 16 core machine, deep scans achieve 90 percent plus cpu utilisation

mcp supply chain risk scenario:

Hypothetical attack (2026):

1 Attacker compromises @modelcontextprotocol/server-filesystem@2.0.1 on npm

2 Adds backdoor that exfiltrates file paths and environment variables

3 Package is downloaded by 10,000+ developers using Claude Desktop

three scan profiles of bumblebee:

baseline (recurring lightweight inventory):

scans language chain tools, editor extensions, browser extensions and mcp configs, daily scans basically using cron(linux), launchd(macOS) etc

use case: on going fleet inventory for good state

typical scan time: 5-15 seconds

project (for the development workspace)

scans development directories

use case: active projects

scan time: 30-60 seconds

deep scan incident response

scans root paths , including broad roots. Refuses to ru without the root specifications. When advisory drops, it gives on demand incident response

Maintained Threat Intelligence Catalogs

Perplexity maintains curated exposure catalogs in the threat_intel/ directory, built from public threat intelligence and updated via community PRs:

- **Laravel Lang (2026-05): Typosquatting attack on Laravel localization packages**
- **PyPI Fabric Compromise (2024-08): Backdoored fabric Python package**
- **XZ Utils Backdoor (2024-03): The infamous liblzma supply chain attack**
- **NPM Malware Campaigns (2024-2026): Ongoing npm registry compromise patterns**

These catalogs are assembled using Perplexity Computer and validated by security researchers, providing a continuously updated knowledge base for incident response.

bumble bee pipeline:'

types in software supply chain security:

dynamic analysis - (we focus on this)

vulnerability scanning

known bad lists

static analysis

inventory and signing